Primer: MPEC for Dynamic Discrete Choice

Su and Judd (2012)

econirl package documentation

1 Context and Problem

Rust's NFXP algorithm (Rust, 1987) estimates DDC parameters by nesting the Bellman solve inside the likelihood maximization. At each candidate θ, the inner loop iterates the contraction mapping $V^{k+1} = T(V^k; \theta)$ until $\|V^{k+1} - V^k\|_\infty < \epsilon$. When the discount factor β is close to 1, the contraction rate approaches unity and convergence slows. At $\beta = 0.9999$, successive approximation (SA) theoretically requires $\lceil \log(\epsilon)/\log(\beta) \rceil = \lceil \log(10^{-10})/\log(0.9999) \rceil =$ 230,258 inner iterations per outer likelihood evaluation. In the logit model, the softmax policy depends on value *differences* which converge faster than absolute levels, so truncated SA can still produce correct MLE estimates. But each outer step still pays the cost of hundreds of SA iterations, and the SA-then-NK polyalgorithm of Iskhakov et al. (2016) replaces each SA call with a handful of Newton steps that cost $O(|\mathcal{S}|^3)$ each.

Su and Judd (2012) eliminate the inner loop entirely by reframing estimation as a single constrained optimization problem. The value function V becomes an explicit decision variable and the Bellman equation $V = T(V; \theta)$ is imposed as a constraint. The constraint is enforced via SQP or an augmented Lagrangian penalty, producing the same MLE in a fraction of the time because zero inner Bellman iterations are needed regardless of β.

2 Model and Notation

The agent observes state $s_t \in \mathcal{S}$ with $|\mathcal{S}| = n$, chooses action $a_t \in \mathcal{A}$, and receives flow utility $u(s_t, a_t; \theta) + \epsilon_{t,a}$ where ϵ is Type I extreme value with scale $\sigma = 1$. The state transitions according to $P(s'|s, a)$ and future payoffs are discounted at rate β.

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\|\mathcal{A}\| = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\|s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

MPEC introduces $\lambda \in \mathbb{R}^n$ (Lagrange multipliers for the Bellman constraint) and $\rho > 0$ (augmented Lagrangian penalty weight, increasing across outer iterations).

3 MPEC Formulation

NFXP (nested).

$$\max_{\theta} \ \mathcal{L}(\theta) \quad \text{where } V(\theta) \text{ solves } V = T(V;\theta) \text{ by contraction.} \tag{1}$$

MPEC (joint).

$$\max_{\theta, V} \ \mathcal{L}(\theta, V) \quad \text{subject to} \quad V = T(V;\theta). \tag{2}$$

The feasible set is identical: any (θ^*, V^*) satisfying the constraint achieves the same log-likelihood value as the NFXP solution. The difference is computational: MPEC optimizes over $(\theta, V) \in \mathbb{R}^{p+n}$ jointly with the constraint enforced by a penalty, rather than eliminating V via a slow fixed-point iteration.

Augmented Lagrangian. The constraint (2) is handled by the augmented Lagrangian

$$\mathcal{L}_\rho(\theta, V, \lambda) = -\mathcal{L}(\theta, V) + \lambda^\top [V - T(V;\theta)] + \tfrac{\rho}{2} \|V - T(V;\theta)\|^2. \tag{3}$$

Each inner subproblem $\min_{\theta,V} \mathcal{L}_\rho$ is solved by L-BFGS-B. The penalty ρ grows by factor γ_ρ each outer iteration until the Bellman residual $\|V - T(V;\theta)\|_\infty < \epsilon_c$.

4 Algorithm

Algorithm 1: MPEC via SQP (Su and Judd, 2012)

Input: Data $\mathcal{D}$, transitions P, discount β, initial θ_0

Warm start: solve $V_0 = T(V_0;\theta_0)$ by value iteration to get feasible (θ_0, V_0)

for $k = 0, 1, \ldots, K_{outer}$ **do**

`// Quadratic subproblem: linearize constraint around current iterate`

Compute $\nabla_{(\theta,V)}\mathcal{L}$ and Jacobian $J = \nabla_{(\theta,V)}[V - T(V;\theta)]$

Solve KKT system for SQP step d^k and multipliers μ^k:

$$\begin{bmatrix} B^k & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} d^k \\ \mu^k \end{bmatrix} = \begin{bmatrix} -\nabla\mathcal{L} \\ -(V - T(V;\theta)) \end{bmatrix}$$

Line search on $\mathcal{L}(\theta + \alpha d^k_\theta,\ V + \alpha d^k_V)$; $\quad (\theta^{k+1}, V^{k+1}) \leftarrow (\theta^k, V^k) + \alpha d^k$

if $\|V^{k+1} - T(V^{k+1};\theta^{k+1})\|_\infty < \epsilon_c$ **then break**

end

Compute standard errors via implicit function theorem at (θ^*, V^*)

Output: θ^*, V^*, standard errors

Implementation notes. The warm start (line 1) initializes V_0 at the Bellman fixed point of the starting θ_0, giving the SQP a feasible interior point. B^k is a BFGS approximation to the Lagrangian Hessian, updated each outer iteration. Gradients of $\mathcal{L}$ and the constraint Jacobian J are computed by JAX automatic differentiation. In practice, the outer SQP uses scipy SLSQP with JAX-supplied gradients.

5 Identification

MPEC and NFXP solve the same MLE problem. At convergence, the constraint $V^* = T(V^*;\theta^*)$ is satisfied exactly (up to feasibility tolerance), so $\mathcal{L}(\theta^*, V^*) = \mathcal{L}_{\text{NFXP}}(\theta^*)$ and both estimators recover the same population parameter vector. Identification requires the same rank condition on the feature matrix $\phi(s,a)$ as NFXP: the matrix Φ formed by stacking $\phi(s,a)$ rows must have full column rank. Scale is normalized by fixing $\sigma = 1$ (logit scale); level is normalized by resetting mileage to 0 on replacement, which anchors RC in the same utility units as the operating cost θ_c. Standard errors are obtained by the observed information formula, evaluating the Hessian at (θ^*, V^*) with the implicit derivative $dV/d\theta = (I - \beta\nabla_V T)^{-1}\nabla_\theta T$ from the implicit function theorem.

6 Simulation

Setting. We use the Rust bus engine with $n = 90$ mileage bins, true parameters $\theta_c = 0.001$, $RC = 3.0$, and discount factor $\beta = 0.9999$. Expert data: 50,000 observations (500 buses $\times$ 100 periods). At $\beta = 0.9999$, successive approximation theoretically requires 230,258 inner iterations per outer likelihood call to reach tolerance 10^{-10}; we cap NFXP-SA at 500 to benchmark each method under the same setting.

Results. All three estimators recover the same MLE: OC = 0.001233 and RC = 3.0295 (Bellman residual 1.9e-10). The logit gradient's level-invariance in V means truncated SA still finds the correct optimum, so all methods converge. The difference is speed. MPEC-SQP enforces the Bellman equation as an equality constraint in a single outer optimisation over $(\theta, V) \in \mathbb{R}^{p+n}$, running no inner iterations, and converges in 0.6s. NFXP-SA takes 4s (7x slower). NFXP-NK (Iskhakov et al., 2016) takes 7.4s (12x slower). MPEC does zero inner Bellman iterations regardless of β, which is the source of the speedup.

Behavioral cloning achieves 97.8% policy accuracy (LL = -10537.25), confirming the data are informative but that BC cannot recover replacement costs without exploiting the MDP structure.

Code.

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.mpec import MPECEstimator, MPECConfig
from econirl.preferences.linear import LinearUtility

env = RustBusEnvironment(num_mileage_bins=90, discount_factor=0.9999)
panel = env.generate_panel(n_individuals=500, n_periods=100)
utility = LinearUtility(env.feature_matrix, env.parameter_names)

mpec = MPECEstimator(MPECConfig(solver="sqp"))
result = mpec.estimate(panel, utility, env.problem_spec, env.transition_matrices)
print(result.parameters)   # [OC, RC] in ~0.6s
```

Run `python mpec_run.py` in this folder to regenerate all results. Estimator source: `src/econirl/estimation/mpec.py`.

References

Fedor Iskhakov, Jinhyuk Lee, John Rust, Bertel Schjerning, and Kyoungwon Seo. Comment on "Constrained optimization approaches to estimation of structural models". *Econometrica*, 84(1):365–370, 2016.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5): 999–1033, 1987.

Che-Lin Su and Kenneth L. Judd. Constrained optimization approaches to estimation of structural models. *Econometrica*, 80(5):2213–2230, 2012.